

Clean Code

Dossier Technique

FUGIER Corentin SEIGNOVERT Maxime VERGUET Romain ZIJP QUENTIN DA SILVA Alexy



SOMMAIRE

Dossier Technique.....	1
Technos du Projet.....	3
1. Backend.....	3
2. Frontend.....	3
3. Base de Données.....	3
Fonctionnalités de notre application OurSpace :.....	4
Processus d'installation et de déploiement :.....	5
Processus d'installation Back-end :.....	5
Processus d'installation Front-end :.....	5
Explications sur la configuration des tests unitaires et automatisés.....	6
ArticleServiceTests.cs.....	6
GetArticlesAsync_ReturnsListOfArticles.....	6
GetArticleByIdAsync_ReturnsArticle.....	6
CreateArticleAsync_CreatesArticle.....	7
CommentServiceTests.cs.....	7
GetCommentByArticleIdAsync_ReturnsListOfComments.....	7
AddCommentAsync_AddsComment.....	7
LikeServiceTests.cs.....	8
AddLikeAsync_AddsLike.....	8
DeleteLikeAsync_DeletesLike.....	8
Aspect Clean Code.....	9
1. Utilisation de noms significatifs pour les variables, fonctions et classes.....	9
2. Structure et lisibilité du code.....	10
3. Modularité et séparation des responsabilités.....	10
4. Commentaires et documentation.....	11
5. Évitement de la redondance et simplification de la maintenance.....	11
6. Pratiques de maintenance et bonnes pratiques.....	12
7. Tests et gestion des erreurs.....	12
Retour d'Expérience sur le Projet Clean Code.....	13
Conclusion.....	14

Technos du Projet

1. Backend

- **Technologie** : .NET (C#)
- **Rôle** : Sert de cœur à l'application, gère la logique métier et les points d'accès API.
- **Caractéristiques Clés** :
 - Intégration avec Supabase pour la gestion de base de données.
 - Implémentation de l'authentification et de l'autorisation.

2. Frontend

- **Technologie** : React + TypeScript et Tailwind CSS
- **Rôle** : Fournit une interface réactive et conviviale pour interagir avec le système.
- **Caractéristiques Clés** :
 - Formulaire dynamiques pour ajouter et mettre à jour les articles
 - Rétroaction en temps réel et mise à jour des données.
 - Utilisation de Tailwind CSS pour un style uniforme et propre.

3. Base de Données

- **Technologie** : Supabase
- **Rôle** : Sert de couche de stockage pour toutes les données de l'application.
- **Caractéristiques Clés** :
 - Base de données relationnelle pour stocker les articles, les commentaires et les Likes sur les articles.
 - Fonctionnalités en temps réel pour les mises à jour.
 - Intégration avec des modules d'authentification.

Fonctionnalités de notre application OurSpace :

1. Authentification et Autorisation
 - a. Système de connexion/inscription via Supabase Auth
 - b. Protection des routes privées Gestion des sessions utilisateur
2. Gestion des Articles
 - a. Création d'articles
 - b. Lecture d'articles
 - c. Suppression d'articles (Uniquement ceux de l'utilisateur)
3. Interactions Sociales
 - a. Système de Likes
 - i. Animation du bouton like (framer-motion)
 - ii. Compteur de likes en temps réel
 - iii. Toggle like/unlike
 - iv. État persistant des likes par utilisateur
 - b. Système de Commentaires
 - i. Ajout de commentaires
 - ii. Suppression de ses propres commentaires
 - iii. Affichage chronologique
 - iv. Animation à l'ajout et à la suppression des commentaires (framer-motion)
4. Interface Utilisateur
 - a. Design Responsive
 - i. Adaptation mobile et desktop
 - ii. Layout fluide avec Tailwind CSS
 - b. Composants Interactifs
 - i. Barre de recherche d'articles
 - ii. Skeletons de loading (dans la homePage et la page d'article)
 - iii. Barre de progression de lecture (en haut des page d'articles)
5. Performance et UX
 - a. Optimisations
 - i. Lazy loading des images des articles
 - ii. Rafraîchissement automatique des données
 - iii. Gestion des états de chargement
6. CI/CD Pipeline
 - a. GitHub Actions avec :
 - i. Tests automatisés
 - ii. Build
 - iii. Déploiement automatique
 - iv. Support multi-versions Node.js

Processus d'installation et de déploiement :

Processus d'installation Back-end :

Prérequis :

- Visual Studio / Rider
- [Code](#)

Lancement de l'app avec '*dotnet run*'

Ouvrez votre navigateur et accédez à <http://localhost:5274/swagger/index.html> pour voir la documentation Swagger.

La base de données est une base Postgres hébergée avec Supabase.

Processus d'installation Front-end :

Prérequis :

- Visual Studio Code / WebStorm
- [Code](#)
- Le code Back-End lancer

Lancement de l'application

Installation des dépendances

```
npm install
```

Lancement de l'application

```
npm run dev
```

Lancement des tests end to end

```
npx cypress open
```

Explications sur la configuration des tests unitaires et automatisés

Nous avons fait des tests unitaire pour chaque service qui test les different méthode, idéalement on aurait voulu faire chaque test non dépendant par exemple :

Pour le test `GetArticleByIdAsync_ReturnsArticle` il aurait fallu faire sans la création d'un article mais comme il n'y a qu'une base on ne peut pas être sûr que l'article ne se fera pas supprimer. Pour résoudre ce problème il aurait été possible d'avoir une base de test qui se réinitialise à chaque début de test.

Présentation des tests :

ArticleServiceTests.cs

GetArticlesAsync_ReturnsListOfArticles

Description : Teste si la méthode `GetArticlesAsync` renvoie une liste d'articles.

Assertions :

- Le résultat n'est pas nul.
- Le résultat est de type `List<ArticleDTO>`.

GetArticleByIdAsync_ReturnsArticle

Description : Teste si la méthode `GetArticleByIdAsync` renvoie l'article correct par ID.

Préparation :

1. Créer un nouvel article.

Assertions :

- Le résultat n'est pas nul.
- L'ID du résultat correspond à l'ID de l'article créé.

Nettoyage :

- Supprimer l'article créé.

CreateArticleAsync_CreatesArticle

Description : Teste si la méthode `CreateArticleAsync` crée un nouvel article.

Préparation :

1. Créer un nouvel article.

Assertions :

- L'ID de l'article créé est supérieur à 0.

Nettoyage :

- Supprimer l'article créé.

CommentServiceTests.cs

GetCommentByArticleIdAsync_ReturnsListOfComments

Description : Teste si la méthode `GetCommentByArticleIdAsync` renvoie une liste de commentaires pour un ID d'article donné.

Assertions :

- Le résultat n'est pas nul.
- Le résultat est de type `List<CommentDTO>`.

AddCommentAsync_AddsComment

Description : Teste si la méthode `AddCommentAsync` ajoute un nouveau commentaire.

Préparation :

1. Créer un nouvel article.
2. Ajouter un nouveau commentaire à l'article.

Assertions :

- Le commentaire ajouté n'est pas nul.
- L'ID du commentaire ajouté est supérieur à 0.

Nettoyage :

-
- Supprimer le commentaire ajouté.
 - Supprimer l'article créé.

LikeServiceTests.cs

AddLikeAsync_AddsLike

Description : Teste si la méthode `AddLikeAsync` ajoute un nouveau like.

Préparation :

1. Créer un nouvel article.
2. Ajouter un nouveau like à l'article.

Assertions :

- Le like ajouté n'est pas nul.
- L'ID du like ajouté est supérieur à 0.

Nettoyage :

- Supprimer le like ajouté.
- Supprimer l'article créé.

DeleteLikeAsync_DeletesLike

Description : Teste si la méthode `DeleteLikeAsync` supprime un like.

Préparation :

1. Créer un nouvel article.
2. Ajouter un nouveau like à l'article.

Assertions :

- Le like est supprimé avec succès.

Nettoyage :

- Supprimer le like ajouté.
- Supprimer l'article créé.

Aspect Clean Code

1. Utilisation de noms significatifs pour les variables, fonctions et classes

Nous avons adopté des conventions explicites pour assurer la clarté et la cohérence dans tout le projet.

Règles suivies :

- **Variables locales dans les fonctions** : Nom en `camelCase` pour refléter leur portée limitée.
- **Variables privées au niveau des classes** : Préfixées par un underscore (`_camelCase`) pour indiquer leur visibilité.
- **Fonctions et propriétés publiques** : Utilisation de la convention `PascalCase` pour respecter les normes en C# et améliorer la lisibilité.
- **Fonctions privées** : Nom en `camelCase`, en cohérence avec leur visibilité limitée.

Code exemple :

```
public async Task<IActionResult> CreateArticle([FromBody] ArticleDTO
article)
{
    if (string.IsNullOrEmpty(article.Title) ||
string.IsNullOrEmpty(article.Content))
        return BadRequest("Le titre et le contenu sont requis");

    await _articleService.CreateArticleAsync(article);
    return CreatedAtAction(nameof(GetArticleById), new { id = article.Id },
article);
}
```

2. Structure et lisibilité du code

L'organisation du projet suit une architecture en couches, séparant la logique métier, les contrôleurs API, et les interactions avec la base de données.

-
- **Couches définies :**
 - **Controllers** : Gèrent les requêtes HTTP.
 - **Services** : Contient la logique métier.
 - **Repositories** : Fournissent des méthodes d'accès aux données via Supabase.
 - **Lisibilité assurée par les pratiques suivantes :**
 - Indentation cohérente.
 - Utilisation de fonctions courtes et concises, chacune remplissant une responsabilité claire.
 - Suppression du code mort ou inutilisé.

Exemple : Organisation des services dans le `Program.cs` pour injecter les dépendances nécessaires :

```
builder.Services.AddSingleton<ArticleService>();  
builder.Services.AddSingleton<LikeService>();  
builder.Services.AddSingleton<CommentService>();
```

3. Modularité et séparation des responsabilités

Chaque module remplit une responsabilité unique, ce qui facilite la compréhension, la modification et le test du code.

Exemples :

- `ArticleService` : Gère les opérations liées aux articles.
- `LikeService` : Traite les fonctionnalités de like/unlike.
- `CommentService` : Administre les commentaires.

4. Commentaires et documentation

Nous avons limité les commentaires au strict nécessaire en suivant l'idée que le code bien écrit se documente par lui-même. Cependant, des commentaires ont été ajoutés pour :

-
- Décrire des comportements complexes ou des choix d'implémentation spécifiques.
 - Documenter les routes API dans les contrôleurs.

Exemple de commentaire utile :

```
[HttpGet(Name = "GetArticles")]  
// Route : GET /articles - Retourne tous les articles disponibles  
public async Task<IActionResult> GetArticles()  
{  
    var articles = await _articleService.GetArticlesAsync();  
    return Ok(articles);  
}
```

5. Évitement de la redondance et simplification de la maintenance

Pour réduire la redondance, nous avons utilisé :

- **L'injection de dépendances**, qui élimine la duplication de code et améliore la testabilité.
- Des fonctions utilitaires réutilisables pour des opérations courantes, comme la validation des données ou le formatage des réponses HTTP.

Exemple de validation centralisée :

```
if (string.IsNullOrEmpty(article.Title) ||  
    string.IsNullOrEmpty(article.Content))  
    return BadRequest("Le titre et le contenu sont requis");
```

6. Pratiques de maintenance et bonnes pratiques

- **Configuration centralisée** : Les paramètres de l'application, tels que la configuration CORS et les clés API, sont définis dans un seul fichier ([Program.cs](#)), facilitant ainsi les modifications futures.

-
- **Async/Await** : Utilisation cohérente pour éviter le blocage des threads et améliorer les performances.
 - **Swagger** : Génération automatique de documentation API pour faciliter les tests et l'intégration avec d'autres systèmes.

7. Tests et gestion des erreurs

Les tests suivent les principes AAA (Arrange, Act, Assert) pour garantir leur clarté et leur robustesse.

Exemple d'un test unitaire pour la récupération d'un article par ID :

```
[Fact]
public async Task
GetArticleByIdAsync_ShouldReturnArticle_WhenArticleExists()
{
    // Arrange
    var articleId = 1;
    // Act
    var result = await _articleService.GetArticleByIdAsync(articleId);
    // Assert
    Assert.NotNull(result);
    Assert.Equal(articleId, result.Id);
}
```

Retour d'Expérience sur le Projet Clean Code

Points Positifs :

- **Stack Technologique Moderne** : L'utilisation de technologies récentes a considérablement amélioré l'efficacité du développement et permis l'intégration de fonctionnalités avancées, notamment les mises à jour en temps réel.
- **Conception Modulaire** : La séparation claire des responsabilités entre le backend, le frontend et la base de données a simplifié le développement et facilité la maintenance.
- **Scalabilité** : L'architecture adoptée est conçue pour évoluer facilement, ce qui permet d'envisager des améliorations futures, comme l'ajout de nouveaux rôles utilisateur ou l'enrichissement des fonctionnalités existantes.

Défis Rencontrés :

- **Problèmes avec les Tests CI/CD** : L'intégration des pipelines de tests dans la chaîne CI/CD a posé des problèmes, notamment des échecs intermittents. Cela a nécessité une configuration précise et des ajustements réguliers.
- **Temps Imparfaitement Alloué** : Le temps imparti pour réaliser le projet était court, ce qui a imposé une pression importante pour respecter les deadlines tout en maintenant un haut niveau de qualité.
- **Gestion des Versions** : Des conflits liés à des modifications simultanées sur les mêmes fichiers ont entraîné des erreurs et retardé l'avancement de certaines fonctionnalités.
- **Travail à Distance** : Collaborer à distance a ajouté des contraintes, notamment en matière de communication et de synchronisation des tâches, ce qui a parfois ralenti le développement.

Conclusion

Ce projet a été une expérience enrichissante, permettant d'appliquer de manière concrète les concepts de **clean code**. Il a non seulement renforcé nos compétences techniques mais également notre capacité à résoudre des problèmes complexes dans un environnement de développement collaboratif. Cependant la deadline du projet a été un peu courte.